

Citi MLOps: Dataset Orchestration and Finetune Platform

Document type: Project plan and architecture (living document)

Status: D

Executive summary

This project delivers an orchestrated, API-first product for: (1) multiformat data ingestion (upload, S3, blob), (2) data preparation and dataset creation in our control plane (chunking, cleaning, row generation, train/test/val split), (3) LLM-assisted steps via GSSP GS (treated as an OpenAI-compatible LLM API for calls only), (4) Stellar to register and store data produced by the pipeline (Stellar does not create datasets; we build datasets first, then register/store artifacts in Stellar as required), (5) finetune and evaluation via the appropriate Citi services (endpoints and ownership per integration contract; Stellar is not the dataset-creation system), and (6) a Model Garden / marketplace UI so users can select, preconfigure, and inspect where base and finetuned models live-without this platform hosting model weights or running training compute locally.

A. Problem statement, scope, and corrected assumptions

A.1 In scope

- Orchestration (BFF, workflows, state, retries, audit context).
- Ingestion from direct upload and object storage (S3, Azure Blob, and equivalents per Citi standards).
- Data preparation owned by this product: chunking, normalization, row construction, validation, optional retrieval-only vectorization when a recipe explicitly needs RAG-style preparation (not the default for every recipe).
- GSSP GS for LLM calls only (OpenAI-API-style: chat/completions; prompts, tools if supported). All other preparation logic-scheduling, chunk boundaries, schema assembly, splitting-lives in orchestration and/or Citi data services outside GSSP.
- Stellar: Registering and storing data only (per Citi Stellar contract). Stellar is not used to create or define training datasets. We build datasets in our pipeline (rows, files, manifest); then we register and store the resulting artifacts in Stellar for downstream use. Finetune and run status for training jobs: follow the Citi service map (Stellar, another gateway, or both-separate from ?dataset creation?). We own train / test / val split in orchestration, before or as we prepare what gets registered in Stellar.
- Evaluation via internal Evaluation API; surface results in metrics and dashboards.
- Auth context propagated on every call: Correlation ID, SOEID, Client ID, Coin JWT (or successor), for approvals and audit.

A.2 Out of scope (for core product line)

- Hosting model weights, serving inference at scale, or local GPU training (Unslotoh-style) for this line.
- Treating GSSP as a full data-prep or ETL engine: it is not; it is LLM API only.

A.3 GSSP GS (correction)

- GSSP GS = LLM API surface (analogous to OpenAI API): used for generation, transformation, classification labels, JSON filling, etc., given our prepared prompts and inputs.
- We own: ingestion, chunking, cleaning, dataset creation (all row assembly), orchestrated multi-step pipelines, row schemas, splits, quarantine, and export formatting that is then registered/stored in Stellar (Stellar does not create the dataset). GSSP is invoked where a step needs an LLM; it does not replace a data platform.

A.4 Stellar: register / store only; not dataset creation (correction)

- Stellar is for registering and storing data (handles, objects, or metadata the API defines). It does not create or build training datasets.
- Dataset creation (examples, JSONL, tables, splits, validation) is entirely in orchestration and connected services; outputs are then registered/stored in Stellar as needed for lineage and handoff.

- Train / test / (val) split is always our responsibility in the preparation pipeline, before or as we produce artifacts that are registered in Stellar-Stellar does not perform splitting or row generation.
- Typical order: (1) ingest and prepare in our system, (2) build and split the dataset, (3) register/store the resulting artifact(s) in Stellar, (4) invoke finetune and eval per the Citi APIs (which may or may not be the same system as the Stellar register/store path-see integration contract).

A.5 Model garden / marketplace (correction, no hosting)

- We do not host models in the platform.
- We must still provide a Model Garden (marketplace) experience in the UI:
- Discover and list base and finetuned models from all integrated model registries and catalogs (internal catalogs, Hugging Face where allowed, Citi model stores, etc.); Stellar is not assumed to be a model registry-use the Citi model integration map (Stellar may only handle data register/store).
- Select a base or finetuned model for preconfiguration of a finetune job (where policy allows).
- View metadata: where the model lives (registry, URI, ID), lineage (e.g. parent base), version, compliance tags.
- Button-driven flows: "Choose model", "View in registry" (link-out), "Use in pipeline."
- The BFF aggregates read-only (and where permitted, action) calls to registry APIs; no weights through this app.

A.6 Modalities and recipe families

- Multiformat text (PDF, DOC, etc.), image (as input to prepared text or multimodal rows per policy).
- Recipes include: Q&A, summarization, sentiment, classification, structured extraction, completion, conversational / multi-turn, multi-model / multi-turn conversation construction (orchestrated calls to GSSP with different model_ids per turn where supported).
- Ingestion from S3 and blob in addition to upload.

A.7 Local compute and "no model ops"

- No colocated finetune compute in-app.
- Yes to Model Garden UI and job configuration that point to external training and models (Citi finetune/training service per contract, not conflated with Stellar's register/store role).
- Vectorization only when a recipe requires retrieval in the preparation path; not required for all finetune datasets.

B. Phased plan of action (implementation)

B.1 Phase 0 - Decisions and guardrails

- ADRs: BFF language, Stellar + GSSP + Ingestion contracts (OpenAPI or internal specs).
- PII, retention, regions, and which registries are allowed for Model Garden.

B.2 Phase 1 - Identity and request context

- Middleware: validate Coin JWT (or delegate to Citi IdP), bind SOEID, Client ID, Correlation ID to Run and outbound HTTP.
- Structured logging and trace propagation.

B.3 Phase 2 - Ingestion

- Upload and S3 / blob connectors; Ingestion service for chunking and provenance; return document_id, chunks (or job handles).

B.4 Phase 3 - Data preparation (platform-owned, GSSP as LLM only)

- Recipe engine: steps for cleaning, row generation (calls GSSP with our prompts/inputs), validation.
- Split: in our pipeline after (or as part of) dataset materialization and before (or as part of packaging for) register/store in Stellar; persist split boundaries, manifests, and references used by finetune and eval APIs.

B.5 Phase 4 - Stellar and downstream training handoff

- Stellar: Register and store prepared artifacts per contract (no dataset creation on Stellar).
- Finetune (or equivalent) job submission to the Citi training/finetune API per service map (not assumed to be ?Stellar creating datasets?): model selection from Model Garden + hyperparameters + identifiers for the register/stored data and splits as required by that API.
- Status polling, error handling, idempotency for each integrated endpoint.

B.6 Phase 5 - Model Garden (UI + BFF aggregation)

- Model registry connectors (incremental per Citi: primary internal catalog first, then others; separate from Stellar data register/store unless contract ties them).
- List, filter, detail pages; lineage and "where it lives"; select for job.

B.7 Phase 6 - Evaluation and observability

- Evaluation API after finetune (or on demand).
- Dashboards: pipeline health, split stats, register/store and finetune-job duration (per integrated services), eval metrics, comparison across runs (same correlation / SOEID filters for audit views).

B.8 Phase 7 - Hardening

- HITL quarantine for bad rows, SLOs, runbook for failed Stellar or GSSP steps.

C. Deviance from the upstream Unsloth project

| Area | Reuse | Replace / new |

|-----|-----|

Usability: treat upstream as a head-start for UX and engineering patterns, not as the execution engine for Citi. Reusability is strongest in frontend; weakest in GPU training and local model paths.

D. End-to-end tech stack (suggested)

- UI: React, Vite, existing component patterns; add routes for Model Garden, Runs, Sources.
- BFF: FastAPI (or Citi-mandated stack) for orchestration, not for ML training.
- Persistence: PostgreSQL (runs, steps, recipe version, split metadata, Stellar ids, model registry cache pointers).
- Queue: async workers for long Ingestion / Stellar / GSSP batch jobs.
- Object storage: S3/Blob; signed URLs where applicable.
- Observability: OpenTelemetry, correlation id on all spans.

E. Studio style vs. agent style

- Primary: Studio - recipes, runs, data sources, Model Garden, register/store and finetune-job status (per Citi service map), eval dashboards.
- Secondary: Assisted flows (wizards, prompt drafts via GSSP) without opaque autonomous agents for regulated use.

F. Visualization

- Run detail: step timeline, GSSP call summaries (no raw secrets), Stellar register/store status (and other service job states as wired), split summary (counts per split).
- Model Garden: cards and tables, lineage graph (lightweight).
- Eval: metrics (task-dependent), trends, compare two finetune runs.
- Reuse Unsloth-style card and table layout; add Citi-appropriate chart components.

G. Data cleaning and extra processing (optional steps)

- PII / redaction, dedupe, length limits, multimodal normalization, multi-turn repair, schema validation, quarantine queue.

H. Glossary (this document)

- GSSP GS: LLM API only (OpenAI-style); not a full data platform.
- Stellar: Registering and storing data only; not dataset creation, row generation, or splitting. We own dataset creation and train/test/val split in orchestration, then register/store in Stellar as required. Finetune and eval use the Citi endpoints defined in the integration contract (not conflated with Stellar's register/store role unless the same platform exposes multiple roles explicitly).
- Model Garden: UI + BFF aggregation to list/select/view external models; no weight hosting in-app.

I. Next steps to maintain this document

- Link Stellar and Ingestion API versions when frozen.
- Add sequence diagrams in a `docs/citi/diagrams/` folder if useful.
- Update Phase checkboxes in project management tool of record; keep this file as the narrative source of truth.

This document was produced for the Citi fork / orchestration product. It may deviate from upstream Unsloth documentation, which targets local Unsloth training and Studio features.